Aarya Arban
T11 – 05
Assignment No. 5

Aim: Database Integration with UI

Theory:

Database integration with the User Interface (UI) is a critical part of application development. It allows apps to store, retrieve, update, and delete data persistently. This enables dynamic interaction between users and data.

Technologies Used

For this assignment, students may choose any of the following:

- **Sqflite** (SQLite plugin for Flutter)
- **Firebase** (Cloud-hosted NoSQL database)
- Any other relational or NoSQL database

Basic Steps for Database Integration

1. Setup and Configuration

- Add the database package to your project dependencies (e.g., sqflite or firebase_core, cloud_firestore).
- Initialize the database connection during app startup.

2. Create a Database or Collection/Table

- For **Sqflite**, define a local SQLite database schema with tables and columns.
- For **Firebase**, define Firestore collections and documents.

3. Create the UI

- Design forms or input fields in the UI using frameworks like **Flutter**, **React**, or **Android XML**.
- Bind form fields to data models.

4. Implement CRUD Operations

a. Create (Insert)

- Save data from the UI into the database.
- For example, on form submission, insert data using SQL commands or Firebase APIs.

b. Read (Retrieve)

- Fetch data from the database to display in the UI.
- Use queries to retrieve all records or specific filtered data.

c. Update

- Allow users to edit existing data in the UI.

- Send update commands to the database with new values.

d. Delete

- Allow users to delete records via buttons or gestures.
- Delete the record from the database using the appropriate command.

5. Display Updated Data

- Use state management or reactive updates to reflect database changes in the UI in real-time (especially with Firebase).

Sample Code (Using Sqflite – Flutter)

dart
CopyEdit

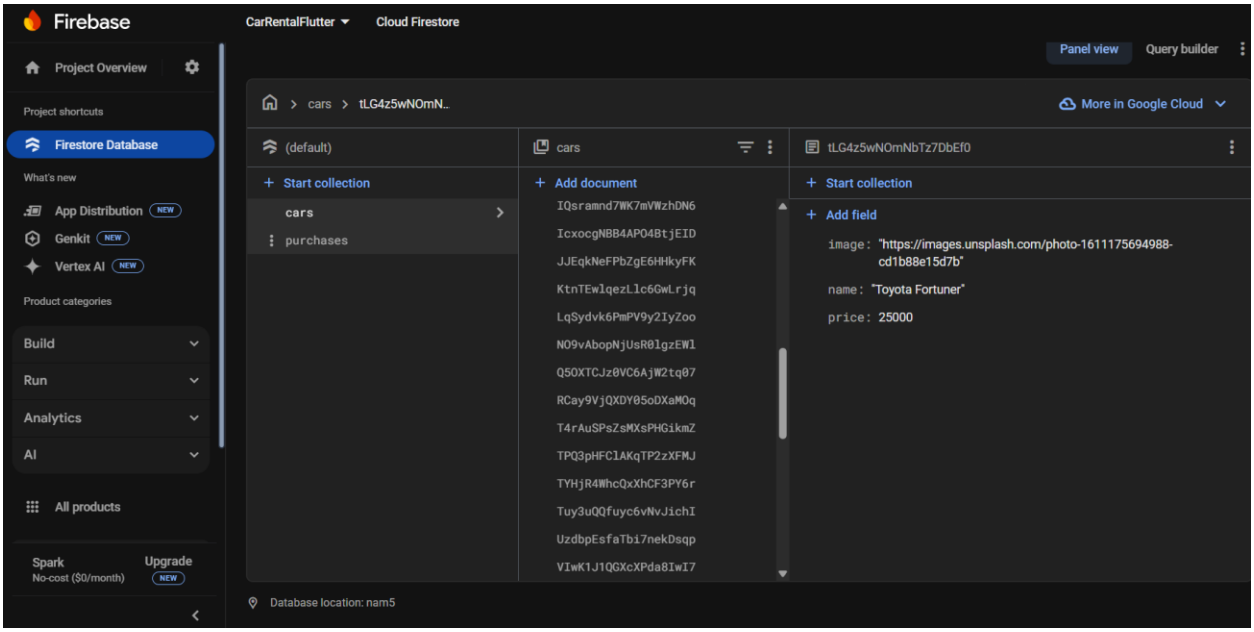
```
// Insert data
await db.insert('students', {'name': 'Alice', 'age': 20});

// Fetch data
List<Map<String, dynamic>> students = await db.query('students');

// Update data
await db.update('students', {'age': 21}, where: 'name = ?', whereArgs: ['Alice']);

// Delete data
await db.delete('students', where: 'name = ?', whereArgs: ['Alice']);
```

Output:



Conclusion

Database integration allows apps to provide meaningful, persistent, and interactive user experiences. Using CRUD operations, developers can manage user data efficiently and securely.